



Universidade do Minho
Escola de Engenharia
Mestrado em Inteligência Artificial

Unidade Curricular de Aprendizagem Profunda

Ano Letivo de 2025/2026

Classificação Textual: Reconhe- cimento de Padrões de LLM's

Carlos da Mota Bergueira (pg11605@alunos.uminho.pt)
Diego Jefferson Mendes Silva (pg59999@alunos.uminho.pt)
Filipa Araújo Pereira (pg42201@alunos.uminho.pt)
Rui Manuel Martins Marques Rodrigues (pg7942@alunos.uminho.pt)

29 de março de 2026

Data de Receção	
Responsável	
Avaliação	
Observações	

Classificação Textual: Reconhecimento de Padrões de LLM's

Carlos da Mota Bergueira (pg11605@alunos.uminho.pt)

Diego Jefferson Mendes Silva (pg59999@alunos.uminho.pt)

Filipa Araújo Pereira (pg42201@alunos.uminho.pt)

Rui Manuel Martins Marques Rodrigues (pg7942@alunos.uminho.pt)

29 de março de 2026

Resumo

O objetivo deste trabalho consiste em desenvolver modelos de aprendizagem supervisionada para classificação de texto, respeitando a restrição de implementação apenas com NumPy e código próprio. Em particular, pretende-se classificar textos segundo a sua origem, distinguindo produção humana de produção automática e, no caso desta última, identificar a família do modelo gerador. Dado que as DNNs exigem entradas numéricas estruturadas, foi necessário construir uma representação tabular dos textos com base em TF-IDF e features manuais. Para avaliar o impacto da não linearidade e da profundidade do modelo, foi implementado um baseline linear regularizado e uma Deep Neural Network multiclasse com mecanismos de controlo de overfitting e melhoria do processo de treino.

Área de Aplicação: .

Palavras-Chave: DNN, BERT, Python, PyTorch, Numpy, .

Índice

1	Introdução	1
1.1	Apresentação do Problema	1
1.2	Motivação e Objectivos	1
2	Dados e Formulação do Problema	2
2.1	Motivação	2
3	Desenvolvimento	3
3.1	Seleção dos Datasets	3
3.1.1	Pré-processamento e Engenharia de Features	3
3.2	Modelo com implementação própria	5
3.2.1	Baseline Linear	5
3.2.2	Deep Neural Network	5
3.2.3	Alternativas Consideradas?	8
3.3	Arquitetura e Desenvolvimento	8
3.3.1	Interface de Utilizador (<i>Front-end</i>)	8
3.3.2	Arquitetura do <i>Back-end</i> e Fluxo de Execução	9
3.3.3	Sequenciamento dos Agentes	11
3.3.4	Respositório do Código	13
4	Resultados e Avaliação	14
4.1	Resultados	14
4.1.1	Eficácia Funcional e Qualidade das Respostas	15
4.1.2	Desempenho e Viabilidade Técnica	15
4.1.3	Potencial de Evolução	15
4.2	Discussão Crítica	15
4.2.1	Pontos Fortes e Eficácia do Sistema	16
4.2.2	Limitações e Desafios Técnicos	16
4.2.3	Análise Comparativa: Local vs. <i>Cloud</i>	16
5	Conclusões e Trabalho Futuro	17
5.0.1	Trabalhos Futuros	17

Lista de Figuras

3.1	Diagrama C4 de Componente.	9
3.2	Diagrama de Fluxo do Agente do Chat.	12
3.3	Diagrama de Sequencia de acionamento textual.	12
3.4	Diagrama de Fluxo de acionamento visual.	13
4.1	Uso do Chat com Somente Texto.	14
4.2	Uso do Chat com Imagem e Texto.	14

Lista de Tabelas

1 Introdução

O objetivo deste trabalho consiste em desenvolver modelos de aprendizagem supervisionada para classificação de texto, respeitando a restrição de implementação apenas com NumPy e código próprio. Em particular, pretende-se classificar textos segundo a sua origem, distinguindo produção humana de produção automática e, no caso desta última, identificar a família do modelo gerador. Dado que as DNNs exigem entradas numéricas estruturadas, foi necessário construir uma representação tabular dos textos com base em TF-IDF e features manuais. Para avaliar o impacto da não linearidade e da profundidade do modelo, foi implementado um baseline linear regularizado e uma Deep Neural Network multiclasse com mecanismos de controlo de overfitting e melhoria do processo de treino.

1.1 Apresentação do Problema

O presente

1.2 Motivação e Objectivos

A motivação deste trabalho reside

2 Dados e Formulação do Problema

O problema foi formulado como uma tarefa de classificação multiclasse. Cada instância corresponde a um texto e a respetiva classe representa a sua origem. O conjunto de dados final resulta da agregação de várias fontes, contendo exemplos de texto humano e de texto gerado por diferentes famílias de modelos. Após agregação e limpeza dos dados, foi criada uma codificação numérica das classes. Para avaliação experimental, os dados foram divididos em treino e teste com preservação aproximada das proporções por classe, através de uma estratégia de divisão estratificada. Esta opção é particularmente importante num cenário multiclasse, pois reduz o risco de enviesamento na avaliação quando existem diferenças de frequência entre classes.

2.1 Motivação

A motivação para este trabalho decorre da atual conjuntura demográfica e imobiliária. O crescimento populacional, aliado à escassez de novas habitações e ao aumento da densidade urbana, tem resultado numa oferta crescente de imóveis com áreas reduzidas. Esta realidade impõe a necessidade de maximizar o aproveitamento do espaço útil através de soluções de design inteligentes.

Neste contexto, observa-se uma procura acentuada por serviços de consultoria capazes de otimizar a habitabilidade destes ambientes. As motivações dos utilizadores são multidimensionais: incluem o desejo de melhorar o conforto e a funcionalidade quotidiana, a necessidade de adaptação a espaços compactos e, predominantemente, o objetivo de valorização patrimonial do ativo imobiliário face a uma eventual transação de mercado. A utilização de Inteligência Artificial surge, assim, como uma ferramenta democratizadora, permitindo que estas soluções de otimização sejam acessíveis a um público mais vasto.

3 Desenvolvimento

O desenvolvimento deste projeto focou-se na criação de um ecossistema robusto e modular, integrando tecnologias de ponta para o processamento de linguagem natural e visão computacional. A implementação estruturou-se em três pilares fundamentais:

3.1 Seleção dos Datasets

Para a construção e treino dos modelos de classificação textual, foram selecionados datasets que englobam uma variedade de textos gerados por humanos e por modelos de linguagem. Estes datasets foram cuidadosamente pré-processados para garantir a qualidade e a relevância das features extraídas, utilizando técnicas como tokenização, remoção de stop words e cálculo de TF-IDF.

3.1.1 Pré-processamento e Engenharia de Features

Os textos foram inicialmente normalizados através de conversão para minúsculas e remoção de caracteres não alfanuméricos, preservando espaços, cada texto bruto t foi transformado numa versão normalizada $\hat{t}$, definida por:

$$\hat{t} = g(t)$$

em que $g(\cdot)$ representa a conversão para minúsculas e a remoção de caracteres não alfanuméricos, preservando apenas sequências relevantes para análise textual. Esta etapa teve como objetivo reduzir ruído e uniformizar a representação lexical dos documentos.

Sobre esta versão limpa foram construídas duas representações TF-IDF complementares. A primeira opera ao nível de palavra, usando n-gramas no intervalo $[1,2]$, de forma a capturar termos isolados e expressões curtas. A segunda opera ao nível de caracteres, usando n-gramas no intervalo $[3,5]$, com o objetivo de captar padrões sublexicais, marcas ortográficas e indícios estilísticos mais finos. Em ambos os casos foi limitado o número máximo de features a 5000.

Para um termo ou n-grama u num documento d , o peso TF-IDF foi calculado como:

$$\text{tfidf}(u, d) = \text{tf}(u, d) \cdot \left(\log \frac{1 + N}{1 + \text{df}(u)} + 1 \right)$$

onde $\text{tf}(u, d)$ representa a frequência relativa do termo u no documento d , $\text{df}(u)$ o número de documentos em que o termo ocorre, e N o número total de documentos do corpus de treino. Após este cálculo, cada vetor foi normalizado com norma L_2 , de forma a reduzir o efeito do comprimento absoluto dos documentos:

$$\text{tfidf}(d) = \frac{v(d)}{\|v(d)\|_2}$$

em que $v(d)$ representa o vetor TF-IDF bruto do documento d . A utilização de TF-IDF a nível de palavra permite capturar conteúdo lexical e pequenas expressões, enquanto a versão baseada em caracteres é útil para modelar padrões ortográficos e estilísticos mais finos.

Para além das representações TF-IDF, foi construído um conjunto de features manuais de natureza estatística e estilística $h(d)$. Entre estas incluem-se comprimento do texto, número de palavras, número de frases, comprimento médio das palavras, diversidade lexical, frequência relativa de pontuação, rácios de maiúsculas, dígitos e espaços, densidade de contrações, uso de repetições de caracteres, palavras em maiúsculas, fragmentos de frase, frases muito longas, palavras de enchimento, conectores formais e pronomes de primeira pessoa. Estas features procuram capturar diferenças de estilo entre texto humano e texto gerado automaticamente. Formalmente:

$$h(d) = [h_1(d), h_2(d), \dots, h_m(d)]$$

onde cada componente $h_j(d)$ corresponde a uma feature estatística ou estilística extraída do documento. Como estas variáveis apresentam escalas distintas, foi aplicada normalização com base nas estatísticas do conjunto de treino:

$$z(d) = \frac{h(d) - \mu}{\sigma}$$

onde μ e σ representam, respetivamente, a média e o desvio-padrão das features calculados no conjunto de treino.

Finalmente, a representação final de cada documento foi obtida pela concatenação dos três blocos de features:

$$x(d) = [x_{\text{word}}(d) \| x_{\text{char}}(d) \| z(d)]$$

em que $x_{\text{word}}(d)$ corresponde ao vetor TF-IDF de palavras, $x_{\text{char}}(d)$ ao vetor TF-IDF de caracteres, e $z(d)$ ao vetor de features manuais normalizadas. Esta representação final constitui a entrada tabular utilizada tanto pelo baseline linear como pela DNN.

3.2 Modelo com implementação própria

O objetivo deste trabalho consiste em desenvolver modelos de aprendizagem supervisionada para classificação de texto, respeitando a restrição de implementação apenas com NumPy e código próprio. Em particular, pretende-se classificar textos segundo a sua origem, distinguindo produção humana de produção automática e, no caso desta última, identificar a família do modelo gerador. Dado que as DNNs exigem entradas numéricas estruturadas, foi necessário construir uma representação tabular dos textos com base em TF-IDF e features manuais. Para avaliar o impacto da não linearidade e da profundidade do modelo, foi implementado um baseline linear regularizado e uma Deep Neural Network multiclasse com mecanismos de controlo de overfitting e melhoria do processo de treino.

3.2.1 Baseline Linear

Como referência inicial, foi utilizado um modelo linear regularizado multiclasse. Este modelo recebe diretamente o vetor de features e aprende uma transformação linear para o espaço das classes, sendo a predição obtida pela classe com maior score. Apesar de não incorporar camadas ocultas nem não linearidades, este baseline permite avaliar até que ponto as features construídas são, por si só, discriminativas.

3.2.2 Deep Neural Network

O modelo principal consiste numa Deep Neural Network feedforward para classificação multiclasse, implementada de raiz em NumPy. A arquitetura é composta por três camadas densas totalmente ligadas, intercaladas com ativações ReLU, camadas de Dropout e uma camada de saída com ativação Softmax.

Arquitetura e Propagação Direta

O vetor de entrada $\mathbf{x}(d) \in \mathbb{R}^F$, que corresponde à representação tabular do documento d construída na secção anterior, é processado sequencialmente por cada camada. A transformação linear na primeira camada densa é:

$$\mathbf{a}_1 = \mathbf{x} W_1 + \mathbf{b}_1, \quad W_1 \in \mathbb{R}^{F \times 256}, \mathbf{b}_1 \in \mathbb{R}^{256}$$

onde W_1 é a matriz de pesos e $\mathbf{b}_1$ o vetor de bias. A não linearidade é introduzida pela função de ativação ReLU, que suprime ativações negativas e atenua o problema do gradiente a desaparecer em redes mais profundas:

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{a}_1) = \max(\mathbf{0}, \mathbf{a}_1)$$

O mesmo padrão repete-se na segunda camada oculta, com dimensão reduzida para 128 unidades:

$$\mathbf{h}_2 = \text{ReLU}(\mathbf{h}_1 W_2 + \mathbf{b}_2), \quad W_2 \in \mathbb{R}^{256 \times 128}$$

A camada de saída projeta as representações internas no espaço das C classes e produz uma distribuição de probabilidades através da função Softmax:

$$\hat{\mathbf{y}} = \text{Softmax}(\mathbf{h}_2 W_3 + \mathbf{b}_3), \quad \hat{y}_c = \frac{e^{z_c - \max_k z_k}}{\sum_{k=1}^C e^{z_k - \max_k z_k}}$$

onde z_c é o score bruto da classe c . A subtração do máximo dos logits antes do cálculo da exponencial é uma estabilização numérica que evita overflow, sem alterar o resultado final.

Inicialização dos Pesos

Os pesos de cada camada foram inicializados com He initialization, adequada à utilização de ativações ReLU:

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right)$$

onde n_{in} é o número de unidades da camada anterior. Esta escolha garante que a variância dos gradientes se mantém aproximadamente constante ao longo das camadas, favorecendo a estabilidade do treino. Os bias foram inicializados a zero.

Função de Perda com Ponderação por Classe

O treino é guiado pela minimização da entropia cruzada multiclasse ponderada. Para um mini-batch de B amostras e C classes, a perda é:

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^B \sum_{c=1}^C w_c y_{ic} \log(\hat{y}_{ic})$$

onde $y_{ic} \in \{0, 1\}$ é o rótulo verdadeiro em formato one-hot e $\hat{y}_{ic}$ é a probabilidade prevista pelo modelo. O fator w_c é o peso da classe c , calculado inversamente proporcional à sua frequência no conjunto de treino:

$$w_c = \frac{N}{C \cdot n_c}$$

em que N é o número total de amostras e n_c o número de amostras da classe c . Esta ponderação penaliza mais os erros nas classes menos representadas, mitigando o viés introduzido por eventuais desequilíbrios na distribuição dos dados.

Regularização por Dropout

Para reduzir overfitting, foram inseridas camadas de Dropout entre as camadas ocultas. Durante o treino, cada unidade é desativada independentemente com probabilidade p_{drop} , e as unidades ativas são reescaladas por $\frac{1}{1-p_{\text{drop}}}$ para preservar a escala esperada das ativações (inverted dropout):

$$\mathbf{h}' = \mathbf{h} \odot \frac{\mathbf{m}}{1 - p_{\text{drop}}}, \quad m_j \sim \text{Bernoulli}(1 - p_{\text{drop}})$$

onde $\odot$ denota o produto elemento a elemento e $\mathbf{m}$ é a máscara binária amostrada aleatoriamente em cada passo de treino. Foram utilizadas taxas $p_{\text{drop}} = 0.3$ após a primeira camada e $p_{\text{drop}} = 0.2$ após a segunda. Durante a inferência, o Dropout é desativado e os pesos não são escalados.

Retropropagação e Regularização L2

A atualização dos pesos segue a descida de gradiente estocástica com mini-batches. O gradiente da perda em relação aos pesos de cada camada densa é calculado pela regra da cadeia e os pesos são atualizados incorporando uma penalização L2:

$$W \leftarrow W - \eta \left(\frac{1}{B} \frac{\partial \mathcal{L}}{\partial W} + \lambda W \right)$$

onde η é a learning rate e λ é o coeficiente de regularização L2 (com valor padrão $\lambda = 10^{-4}$). A penalização L2 reduz a magnitude dos pesos ao longo do treino, favorecendo soluções mais suaves e com menor tendência ao ajuste excessivo. Os bias são atualizados sem regularização.

Early Stopping e Redução Adaptativa da Learning Rate

Para controlar a duração do treino sem requerer um número fixo de épocas, foi implementado early stopping com base na perda de validação. O estado do modelo é guardado sempre que se verifica uma melhoria acima de um limiar mínimo δ_{min} :

$$\mathcal{L}_{\text{val}}^{(e)} < \mathcal{L}_{\text{val}}^* - \delta_{\text{min}}$$

onde $\mathcal{L}_{\text{val}}^*$ é a melhor perda de validação registada até ao momento. Se não se verificar melhoria durante P épocas consecutivas, o treino é interrompido e os pesos correspondentes ao melhor estado são restaurados.

Adicionalmente, foi implementada redução adaptativa da learning rate: se a perda de validação não melhorar durante P_r épocas, a learning rate é reduzida multiplicativamente:

$$\eta \leftarrow \eta \cdot \gamma, \quad \gamma \in (0, 1)$$

Neste trabalho foram utilizados $P = 30$, $P_r = 10$, $\gamma = 0.5$ e $\delta_{\min} = 10^{-4}$. Este mecanismo permite que o modelo explore regiões de menor gradiente sem abandonar prematuramente o processo de otimização.

3.2.3 Alternativas Consideradas?

Foi considerada a utilização de métodos clássicos de *Text Mining*, como sistemas de QA baseados apenas em TF-IDF para pesquisar manuais de design estáticos. No entanto, esta alternativa foi descartada por não permitir a análise multimodal (visão) e ser incapaz de resolver a ambiguidade semântica inerente à linguagem humana.

3.3 Arquitetura e Desenvolvimento

Nesta secção, descreve-se a arquitetura e os componentes técnicos que compõem a solução desenvolvida. O sistema divide-se numa interface de utilizador (*Front-end*) e numa lógica de processamento (*Back-end*) baseada em grafos de estado.

1. *Front-end* (HTML5/Hyperdiv [5]): Gere o estado reativo e o carregamento de imagens.
2. *Back-end* (Python/Hyperdiv [5]): Receciona as mensagens e aciona a pipeline do chatbot.
3. Serviço de Chatbot (LangGraph [4]): Pipeline que efetua a orquestração do chat.
4. Modelos de IA (Ollama [1]): Atuam como o motor de inferência profunda.

3.3.1 Interface de Utilizador (*Front-end*)

Foi desenvolvida uma interface web minimalista utilizando a biblioteca Hyperdiv [5]. Esta interface foi desenhada para proporcionar uma interação fluida, oferecendo as seguintes funcionalidades:

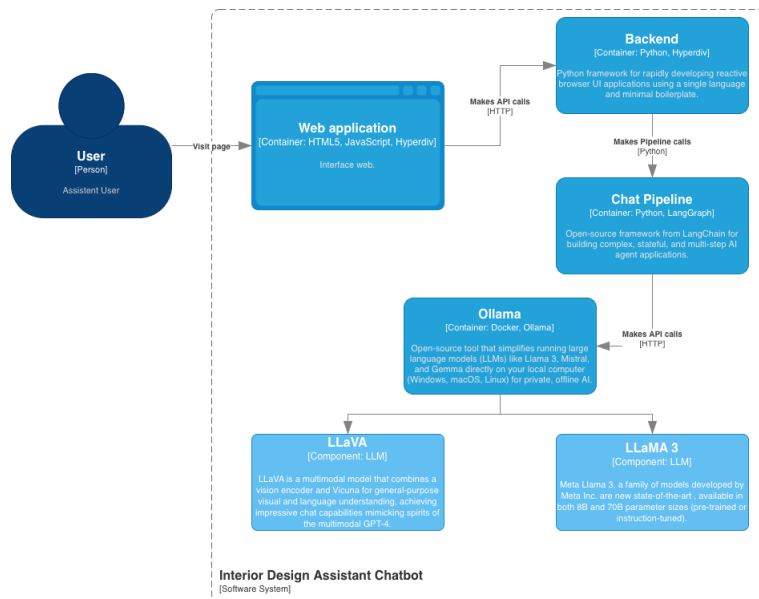


Figura 3.1: Diagrama C4 de Componente.

Gestão de Mensagens: Envio de instruções de texto e visualização do histórico de conversação.

Interação Multimodal: Suporte para anexar ficheiros de imagem, permitindo que o modelo analise o contexto visual do ambiente.

Controlo de Sessão: Opção para limpar o histórico de mensagens e reiniciar o estado da interação.

3.3.2 Arquitetura do *Back-end* e Fluxo de Execução

O núcleo da aplicação reside no back-end, onde cada interação do utilizador é processada por um agente inteligente. A lógica de execução é orquestrada através de uma pipeline construída com a biblioteca LangGraph [4], que permite definir o comportamento do agente como um grafo cíclico de estados.

Ao submeter uma mensagem na interface, os dados são direcionados para o agente, cuja execução está organizada em quatro nós principais:

1. **Roteador:** Este nó atua como o ponto de entrada da pipeline e é responsável pela triagem dos dados submetidos pelo utilizador. A sua função primordial é verificar a presença de componentes multimodais na mensagem:
 - **Deteção de Imagem:** Caso seja identificado um anexo de imagem, o fluxo é direcionado para o Nó Visual, onde será efetuado o processamento de visão computacional.

- **Fluxo Exclusivamente Textual:** Caso não seja detetada qualquer imagem, o sistema aciona o Nó Textual, otimizando o tempo de resposta ao ignorar as etapas de processamento visual desnecessárias.

Esta lógica de ramificação garante que o agente utilize o modelo mais adequado para o tipo de dado recebido, assegurando uma gestão eficiente dos recursos computacionais.

2. **Textual:** Este nó é ativado sempre que a entrada do utilizador consiste exclusivamente em texto. O processamento é delegado ao modelo LLaMA [3], sendo a interação estruturada através de uma instrução de sistema (*System Prompt*) rigorosa que define o comportamento e o domínio de atuação do agente.
 - a) **Configuração do Modelo e Diretrizes:** Para garantir a qualidade e a relevância das respostas, o modelo é instanciado com as seguintes diretrizes permanentes:
 - b) **Persona Profissional:** Atuação exclusiva como assistente especializado em Design de Interiores.
 - c) **Estruturação de Respostas:** Obrigatoriedade de fornecer conselhos práticos e profissionais, estruturados preferencialmente em listas ou passos numerados para facilitar a leitura.
 - d) **Foco na Valorização:** Em cenários de potencial venda do imóvel, o modelo é instruído a priorizar intervenções de elevado impacto visual e custo controlado.
 - e) **Localização Linguística:** Tradução e adaptação obrigatória de todos os outputs para português de Portugal.
 - f) **Gestão de Contexto e Memória:** Uma característica fundamental deste nó é a manutenção da coerência conversacional. A cada nova interação, o sistema injeta o histórico completo da sessão atual na pipeline. Esta abordagem permite que o modelo possua memória de curto prazo, sendo capaz de compreender referências a mensagens anteriores e manter a continuidade lógica do diálogo.
3. **Visual:** Este nó é ativado exclusivamente quando o utilizador partilha ficheiros de imagem. A sua função primordial é a transcrição visual, transformando o conteúdo gráfico em dados textuais detalhados que possam ser interpretados pelos nós subsequentes da pipeline. Diferente do nó textual, este componente opera de forma isolada no que diz respeito ao histórico de conversação. Para maximizar a velocidade de processamento e reduzir a carga computacional, o histórico de contexto não é enviado. O modelo foca-se estritamente na análise imediata da imagem fornecida, garantindo uma resposta ágil e objetiva. O modelo de visão é configurado com uma diretriz específica para assegurar a precisão da análise:
 - **Foco Descritivo:** Atuar como um assistente de design cujo objetivo é listar e descrever todos os elementos visíveis na imagem (mobiliário, iluminação, cores, disposição espacial).

- **Finalidade Técnica:** O output gerado é desenhado para servir de base informativa para o refinamento de sugestões de design que ocorrerá na etapa seguinte.

A função deste nó não é fornecer o conselho final ao utilizador, mas sim converter a percepção visual em linguagem natural. Este "resumo visual" será posteriormente integrado com o contexto histórico no nó final, garantindo que as sugestões de otimização sejam tecnicamente fundamentadas no que foi efetivamente observado na fotografia.

4. **Refinamento:** Este nó constitui a etapa final da pipeline e é ativado sequencialmente após o processamento do Nó Visual. A sua função crítica é a fusão de dados: combinar a descrição técnica da imagem com o histórico da conversa para gerar uma resposta final coerente e personalizada. Integração de Dados e Contexto: O diferencial deste nó reside na sua capacidade de contextualização. O sistema injeta a descrição textual gerada anteriormente através da variável *{image_description}*, permitindo que o modelo "compreenda" o que foi visto na imagem sem a necessidade de processar novamente o ficheiro gráfico. O modelo utiliza um conjunto de instruções (*System Prompt*) focado na qualidade do output final:

- a) **Profissionalismo e Clareza:** O foco principal é a melhoria da clareza e do tom profissional dos conselhos de design de interiores.
- b) **Estruturação Lógica:** Tal como no nó textual, privilegia-se a organização da informação em listas ou passos numerados.
- c) **Estratégia de Valorização:** Reitera-se a prioridade em mudanças de elevado impacto e custo razoável para objetivos de valorização imobiliária.
- d) **Finalização Linguística:** Garante que a síntese final entre a análise da imagem e os conselhos teóricos seja entregue integralmente em português de Portugal.

Ao consolidar a descrição visual com as intenções do utilizador (presentes no histórico), este nó assegura que as sugestões não são apenas genéricas, mas sim diretamente aplicáveis ao ambiente fotografado. O resultado é uma resposta refinada que encerra o ciclo de processamento do agente.

Para uma melhor compreensão da dinâmica de execução e das ramificações de decisão do sistema, apresenta-se a figura 3.2 com o fluxograma do agente de conversa. Este diagrama ilustra a orquestração entre os nós de decisão (roteador), processamento de visão e refinamento textual, evidenciando como o estado é gerido ao longo da pipeline do LangGraph [4].

3.3.3 Sequenciamento dos Agentes

A figura 3.3 contém o diagrama de sequência seguinte ilustra o fluxo de execução quando o utilizador submete uma consulta exclusivamente textual. Este processo destaca a natureza síncrona da comunicação e a recuperação do histórico para garantir a continuidade do contexto.

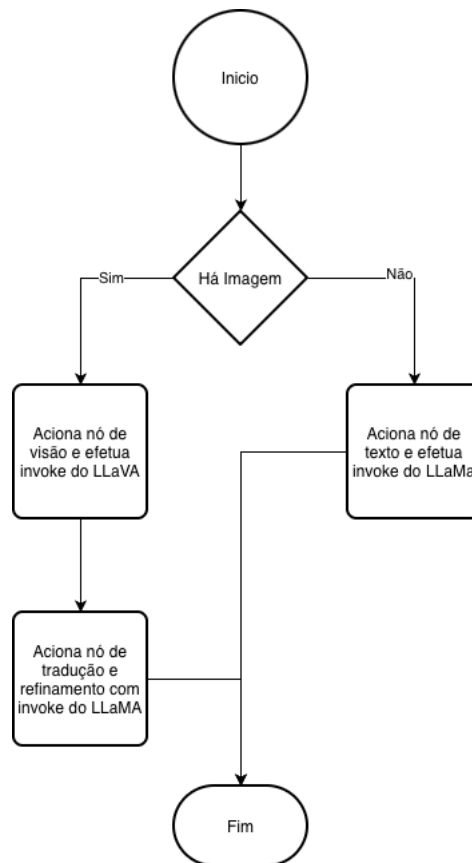


Figura 3.2: Diagrama de Fluxo do Agente do Chat.

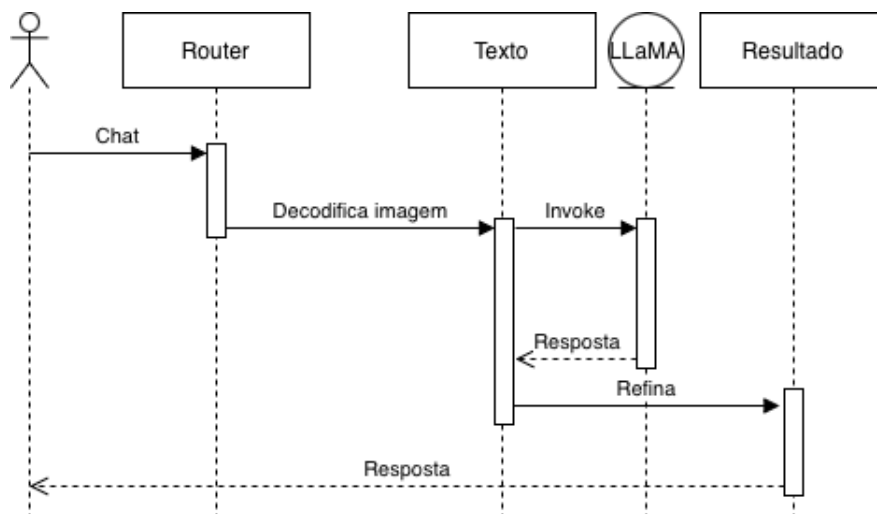


Figura 3.3: Diagrama de Sequencia de acionamento textual.

A figura 3.4 contém fluxo de execução para mensagens que contêm anexos de imagem é mais complexo, exigindo uma orquestração em várias etapas para garantir que o contexto visual seja corretamente interpretado e integrado na resposta final.

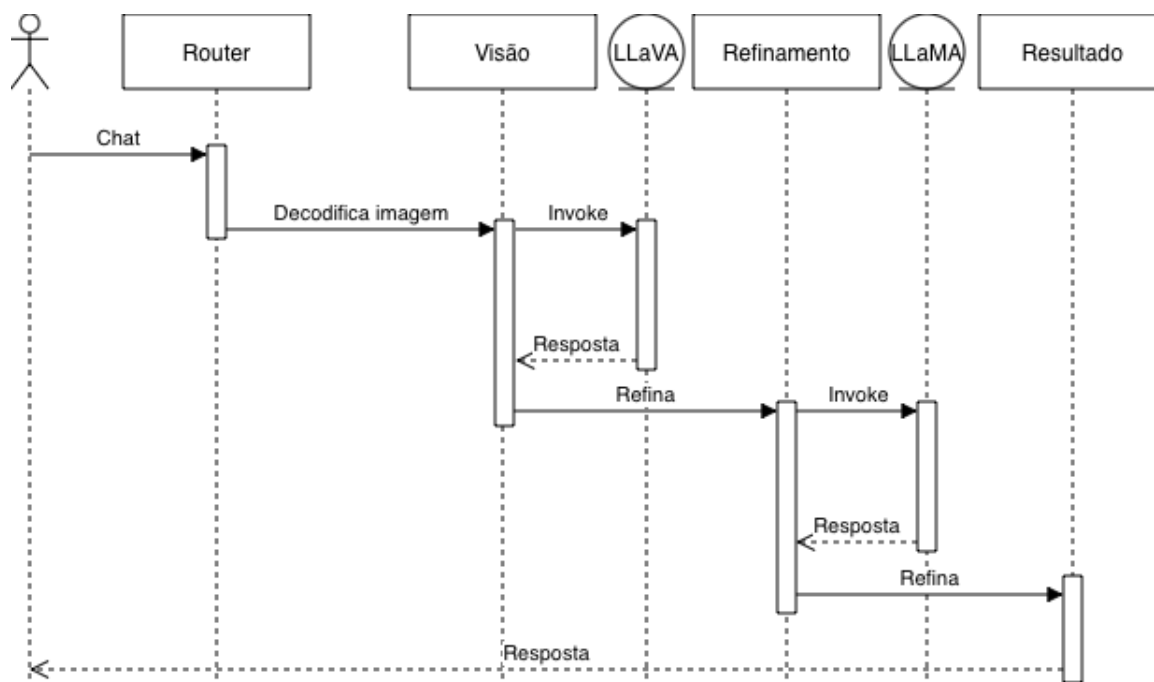


Figura 3.4: Diagrama de Fluxo de acionamento visual.

3.3.4 Respositório do Código

Para efeitos de reprodutibilidade e análise detalhada da implementação, o repositório contendo o código deste trabalho está alojado no GitHub em: <https://github.com/diegojeffersonms/IIA>

4 Resultados e Avaliação

4.1 Resultados

Os testes realizados demonstraram que o assistente inteligente apresenta um comportamento robusto e alinhado com os objetivos propostos. A análise dos resultados foca-se na eficácia funcional e no desempenho técnico do sistema.

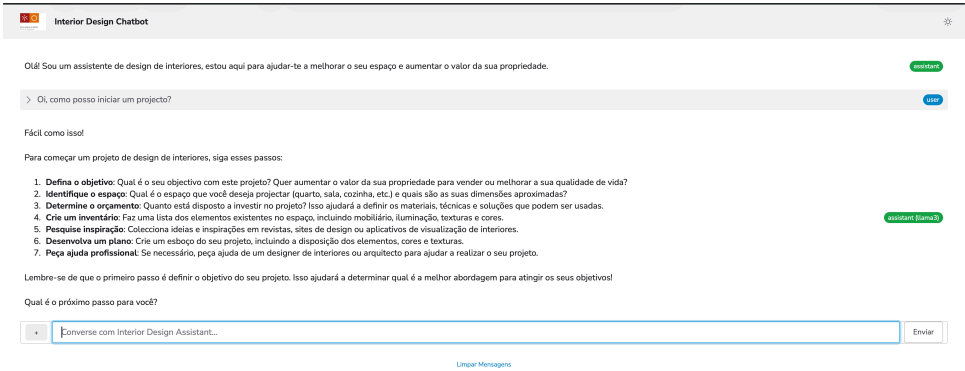


Figura 4.1: Uso do Chat com Somente Texto.

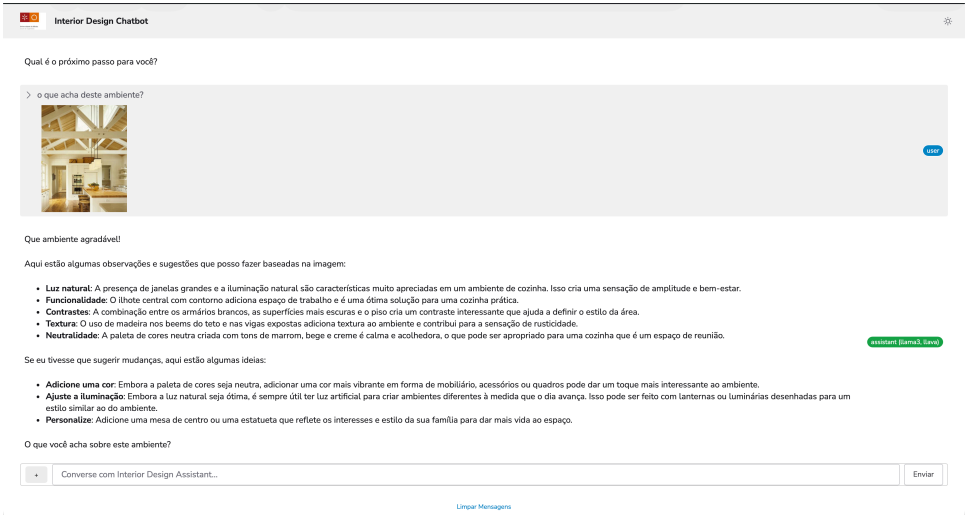


Figura 4.2: Uso do Chat com Imagem e Texto.

4.1.1 Eficácia Funcional e Qualidade das Respostas

O chatbot exibiu uma elevada competência na interpretação de contextos específicos de design de interiores. Observou-se que o sistema foi capaz de:

- **Interpretação Multimodal:** Identificar corretamente elementos em imagens reais, convertendo-os em descrições textuais úteis para o aconselhamento.
- **Pertinência das Sugestões:** Fornecer recomendações logicamente estruturadas, respeitando as diretrizes de otimização de espaço e valorização imobiliária.
- **Manutenção de Contexto:** Responder adequadamente a sequências de perguntas, demonstrando uma gestão de histórico eficaz durante a sessão.

4.1.2 Desempenho e Viabilidade Técnica

Embora o sistema se tenha revelado funcional, o desempenho foi condicionado pela infraestrutura de execução. Devido à utilização do Ollama [1] em ambiente Docker [6] sem aceleração por GPU, registou-se uma latência considerável na geração de respostas. No entanto, este atraso não comprometeu a precisão do modelo, servindo como uma Prova de Conceito para fins académicos bem-sucedido.

4.1.3 Potencial de Evolução

Os resultados indicam que a arquitetura baseada em agentes é altamente escalável. Identificou-se que a precisão e a profundidade das respostas podem ser significativamente potenciadas através da implementação de técnicas de RAG (*Retrieval-Augmented Generation*). A integração de uma base de dados vetorial permitiria complementar o conhecimento do modelo com catálogos técnicos, normas de arquitetura ou tendências de mercado atualizadas, reduzindo a ocorrência de alucinações e aumentando a autoridade das sugestões prestadas.

4.2 Discussão Crítica

A avaliação do protótipo desenvolvido permitiu identificar pontos fulcrais sobre a aplicação de modelos de linguagem locais em contextos de design de interiores. A análise divide-se entre as competências demonstradas pelo sistema e as condicionantes técnicas inerentes à arquitetura escolhida.

4.2.1 Pontos Fortes e Eficácia do Sistema

O sistema demonstrou uma elevada competência na análise multimodal, validando a eficácia da arquitetura em grafos. A capacidade do nó de visão em traduzir elementos espaciais em texto permitiu que o assistente fornecesse conselhos contextuais pertinentes, aproximando-se da percepção humana. Ao nível da experiência de utilizador (UX), a implementação de uma interface reativa com suporte para streaming revelou-se crucial. Esta funcionalidade mitiga a percepção de espera, permitindo que o utilizador acompanhe a geração progressiva da resposta. Adicionalmente, o projeto destaca-se pela sua conformidade ética e privacidade. Ao processar todos os dados localmente via Ollama [1], o sistema adere aos princípios de IA Responsável, garantindo que imagens de espaços privados não são transmitidas para servidores de terceiros.

4.2.2 Limitações e Desafios Técnicos

Apesar dos resultados positivos, foram identificadas limitações que impactam a escalabilidade da solução:

- Observou-se um tempo de resposta significativo (entre 10 a 30 segundos) no processamento de imagens. Este atraso deve-se à elevada carga computacional exigida pelos modelos de visão em hardware local.
- Atualmente, a memória de sessão é volátil. A ausência de uma base de dados dedicada para o histórico de longo prazo implica que o estado da conversação se perca caso a página seja recarregada.
- A execução fluida do sistema impõe requisitos técnicos elevados (mínimo de 8GB a 16GB de RAM), o que pode limitar a acessibilidade da ferramenta em dispositivos menos potentes.

4.2.3 Análise Comparativa: Local vs. Cloud

Comparando esta abordagem com o uso de modelos via API (como o GPT-4o da OpenAI), nota-se um *trade-off* claro. Enquanto uma solução baseada na cloud ofereceria menor latência e uma base de conhecimento mais vasta, comprometeria a soberania de dados e a privacidade das imagens dos utilizadores, além de introduzir custos variáveis por utilização. O modelo local adotado neste trabalho posiciona-se como uma "caixa-preta" segura e privada, priorizando a segurança em detrimento da velocidade instantânea.

5 Conclusões e Trabalho Futuro

O presente trabalho permitiu demonstrar a viabilidade e o potencial da aplicação de Inteligência Artificial Generativa na democratização do acesso a serviços de design de interiores e otimização habitacional. Através do desenvolvimento de um agente inteligente multimodal, foi possível responder à crescente necessidade de maximização de espaços em contextos urbanos de elevada densidade. A arquitetura implementada, baseada na orquestração de grafos de estado com o LangGraph [4] e na execução local de modelos via Ollama [1], revelou-se uma escolha técnica sólida. Esta abordagem não só garantiu a flexibilidade necessária para gerir fluxos complexos (texto e imagem), como também assegurou a total privacidade e soberania dos dados dos utilizadores, um fator crítico quando se lida com registos visuais de ambientes privados. Apesar das limitações identificadas ao nível da latência — decorrentes da execução em hardware sem aceleração dedicada (GPU) — o projeto atingiu com sucesso o estatuto de Prova de Conceito (PoC). O ChatBot demonstrou uma capacidade notável de interpretação visual e raciocínio lógico, fornecendo sugestões pertinentes e personalizadas que cumprem o objetivo de valorização funcional e patrimonial dos imóveis.

5.0.1 Trabalhos Futuros

Como perspetiva de evolução, este projeto estabelece os alicerces para implementações mais sofisticadas. Sugere-se para investigações futuras:

- Implementação de RAG (Retrieval-Augmented Generation) com uso de bases de dados externas para garantir que o assistente utiliza conhecimentos técnicos e normativas arquitetónicas atualizadas.
- Persistência de Longo Prazo com desenvolvimento de uma camada de base de dados para manter o histórico entre sessões, permitindo um acompanhamento contínuo de projetos de renovação.

Em suma, este trabalho evidencia que a IA, quando estruturada de forma modular e ética, pode atuar como uma ferramenta poderosa de apoio à decisão, transformando desafios demográficos e imobiliários em oportunidades de inovação tecnológica.

Referências

- [1] OLLAMA (2024). Ollama: Get up and running with large language models. [Online]. Disponível em: <https://ollama.com> [Acedido em: 11 de janeiro de 2026].
- [2] YOUNG, H. et al. (2023). LLaVA: Large Language and Vision Assistant. [Online]. Disponível em: <https://llava-vl.github.io> [Acedido em: 11 de janeiro de 2026].
- [3] META AI (2024). Introducing Llama 3: The most capable openly available LLM to date. [Online]. Disponível em: <https://ai.meta.com/blog/meta-llama-3/> [Acedido em: 11 de janeiro de 2026].
- [4] LANGCHAIN (2024). LangGraph: Building Language Agents with Graphs. [Online]. Disponível em: <https://www.langchain.com/langgraph> [Acedido em: 11 de janeiro de 2026].
- [5] STEFANOVIC, M. (2024). Hyperdiv: Reactive web UI framework for Python. [Online]. Disponível em: <https://hyperdiv.io> [Acedido em: 11 de janeiro de 2026].
- [6] DOCKER (2024). Docker Documentation. [Online]. Disponível em: <https://docs.docker.com> [Acedido em: 11 de janeiro de 2026].